

Diversifying Fault-Tolerant Systems using LLMs

Arne Vogel¹, Christian Berger¹ und Rüdiger Kapitza¹

Abstract: Fault-tolerant agreement protocols fail if replicas share a common flaw that simultaneously affects more replicas than the tolerable threshold. Therefore replicas should ideally fail independently, which can be achieved through diversification. However, in practice, often the same protocol implementation is shared by all replicas which is not surprising given that the provision of multiple diverse implementations is difficult and highly laborious. This poses a major risk, as a shared protocol implementation is a prime candidate for common bugs due to its complexity.

With DIVERSEFT, we demonstrate how, given a reference implementation, Large Language Models (LLMs) can be utilised for the automated and scalable generation of code that compiles, passes tests, and crucially differs semantically and binary-wise, that can replace code in the reference implementation, thereby significantly reducing diversification costs. We demonstrate the feasibility of diversification of replication protocol implementations using LLMs by diversifying three implementations: PBFT, HotStuff, and Raft, showing how up to 65% of the codebase can be diversified.

Keywords: Byzantine Fault Tolerance, Large Language Models, Crash Fault Tolerance

1 Introduction

In this work, we focus on diversifying the implementations of agreement protocols as otherwise a shared bug in the system could easily violate the limit of f faulty replicas. One of the core methods for providing fault-independent software for replicas is N-version programming [Av85]. It requires the implementation of multiple, diverse versions of a system based on a common specification, ideally by different development teams using different programming languages or development methodologies, to minimize the risk of common-mode errors. As a result, N-version programming is considered prohibitively resource-intensive and is typically not applied to common IT services. Since the seminal work on making Byzantine Fault Tolerance practical for everyday IT services [CL99], the question of how to achieve fault independence in practice without adopting full N-version programming has remained an open topic especially relevant today with distributed ledgers managing billions in value [Be23]. One line of work is *opportunistic* N-version programming, which builds on software heterogeneity for well-established APIs [CRL03]. Despite all previous works, diversification at the protocol level, which is at the heart of replicated systems, has largely been abandoned. This is not surprising, as implementing a fault-tolerant agreement protocol involves highly concurrent code featuring complex communication logic and requires the correct use of various cryptographic methods.

In this paper, we propose DIVERSEFT, which enables the highly automated and scalable diversification of Byzantine Fault Tolerant (BFT) and Crash Fault Tolerant (CFT) agreement

¹ FAU Erlangen-Nürnberg, arne.vogel@fau.de; berger@cs.fau.de; ruediger.kapitza@fau.de

protocol implementations using LLMs. This is achieved by using multiple LLMs to generate diversified implementations based on an original version. We answer questions about at what level diversification should occur, how diversification can be ensured while maintaining functional equivalence and how to ensure actual representational and binary diversification instead of syntactical variations.

We design and implement `DIVERSEFT`, which automates the diversification of Rust-based distributed protocols. Rust is a modern systems programming language widely adopted in academia and industry alike due to its security guarantees [Be24]. To highlight the benefits of `DIVERSEFT`, we automatically diversify Themis (implementing the PBFT algorithm [CL99]), `hotstuff_rs` (implementing the HotStuff algorithm [Yi19]), and Openraft, a CFT replication protocol implementation. `DIVERSEFT` makes diversification practical for BFT and CFT protocols. We demonstrate how to use LLMs to generate protocol changes at function level based on reference implementations. To validate changes, we introduce *safeguards* for functional and diversification correctness.

Contributions. We demonstrate how to use LLMs to generate representation/binary different implementations of complex replication protocols, with up to 65% of the original code being automatically changed. We do this while validating the generated code for functional equivalence with unit tests and protocol conformance tests against the reference implementation ensuring that changes do not break the protocol as well as diversification validation with clone detection and binary difference validation.

Literaturverzeichnis

- [Av85] Avizienis, A.: The N-Version Approach to Fault-Tolerant Software. IEEE Transactions on Software Engineering SE-11 (12), S. 1491–1501, 1985.
- [Be23] Berger, C. et al.: SoK: Scalability Techniques for BFT Consensus. In: 2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC). S. 1–18, 2023, <https://doi.org/10.1109/ICBC56567.2023.10174976>.
- [Be24] Bearer, J. et al.: The Espresso Sequencing Network: HotShot Consensus, Tiramisu Data-Availability, and Builder-Exchange, Cryptology ePrint Archive, Paper 2024/1189, 2024, <https://eprint.iacr.org/2024/1189>.
- [CL99] Castro, M.; Liskov, B.: Practical byzantine fault tolerance. In: OSDI. Bd. 99 1999. 1999, S. 173–186, 1999, https://www.usenix.org/legacy/publications/library/proceedings/osdi99/full_papers/castro/castro_html/castro.html.
- [CRL03] Castro, M.; Rodrigues, R.; Liskov, B.: BASE: Using abstraction to improve fault tolerance. ACM Trans. Comput. Syst. 21 (3), S. 236–269, 2003, <https://doi.org/10.1145/859716.859718>.
- [Yi19] Yin, M. et al.: HotStuff: BFT Consensus with Linearity and Responsiveness. In: Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing. PODC '19, Association for Computing Machinery, Toronto ON, Canada, S. 347–356, 2019, <https://doi.org/10.1145/3293611.3331591>.